Estimating the Unobservable: Per-Request Cost of Blocked Trackers in Firefox

Anonymous Author(s)

Abstract

Firefox's Enhanced Tracking Protection (ETP) blocks third-party tracker requests for hundreds of millions of users, but the network cost it prevents is structurally unobservable: blocked requests never receive a response. We measure the cost landscape of the modern tracker ecosystem using 3,490,824 requests across 3,723 tracker domains from the HTTP Archive, and find that within-domain transfer size spans three orders of magnitude (coefficient of variation 0.94 at the median domain, 3.0 at the 90th percentile). We then present a deployable estimator that predicts per-request transfer size from features observable before a response arrives: an XGBoost model with Tweedie loss that ships as a 500 KB ONNX artifact and beats every lookup-table baseline including exact-path memorization on the rows that path memorization can serve, with non-overlapping bootstrap confidence intervals. The model retains a +28.9% per-request and +10.8 pp aggregation advantage on a 3-month temporal holdout, +25.6% and +5.7 pp at 6 months. In an in-page Firefox crawl of 500 Tranco-sampled pages (Playwright with mobile emulation, 354 surviving pages, 20,750 third-party tracker requests), the model preserves an +11 to +13 percentage-point aggregation advantage over the deployable LUT in domain-correlated browsing (the realistic deployment model where a user repeatedly visits a clustered set of domains). We discuss implications for active IETF/IRTF discussions on tracker measurement, third-party content cost, and quantitative privacy-feature evaluation.

1 Introduction

Enhanced Tracking Protection (ETP) is a privacy intervention deployed across all release channels of Firefox, blocking third-party requests classified as advertising, analytics, social, tag-manager, or consent-provider trackers by the Disconnect list [3]. At the scale of hundreds of millions of daily active users, ETP makes billions of block decisions per day; the privacy dashboard reports a count of trackers blocked but no measure of the network or performance cost prevented. Every blocked request is reported equally: a 258-byte tracking pixel from `bat.bing.com` is indistinguishable from a 170 KB JavaScript SDK from `googletagmanager.com`.

Quantifying what ETP saves is a measurement problem with a structural gap: the request was never sent, so the response was never received. The browser sees only the block-time features (URL, resource type, initiator, HTTP metadata); transfer size, content type, and timing must be predicted. This question is squarely in the network-measurement community's lane: the Privacy Enhancements RG (PEARG) has standing calls for quantitative methodology to compare deployed privacy interventions, the Measurement and Analysis for Protocols RG (MAPRG) hosts deployed-system measurement at scale, and HTTPbis discussions on third-party content cost have nowhere to point for credible per-tracker numbers. Vendor self-reports cover page-level totals [1] or aggregate counts; per-request, per-tracker, per-category measurements grounded in public data have not previously been published.

Contributions. (1) **The cost landscape ETP acts on:** within-domain transfer size spanning three orders of magnitude (CV 0.94 median, 3.0 p90) across 3,723 domains, and per-category byte concentration (a 26-domain Tag-manager category carries 41% of bytes from 6% of requests; §3). (2) **A deployable estimator that beats every deployable LUT we tested**: 500 KB ONNX artifact, 50 μs median single-thread inference, 36.5% MAE reduction over the deployed Domain+type LUT, 25% reduction over the strongest intermediate-granularity deployable LUT we found, 8% reduction even over the non-deployable path-LUT upper bound (§4). (3) **A multi-axis robustness characterization:** per-request advantage held at +33.5%, +28.9%, +25.6% over 1/3/6-month temporal horizons, and cross-browser byte-agreement empirically $\geq$ 97.8% within 5% on a paired Firefox/Chrome fetch (§5, §6). (4) **Concrete inputs to active IRTF discussions** on quantitative privacy-feature evaluation, deployed-system measurement, and third-party content cost (§7).

2 Background and Related Work

ETP and tracker blocking. Firefox ETP, Brave Shields, Privacy Badger, and uBlock Origin make binary block/allow decisions on third-party requests. None reports the network cost of what was prevented; this work closes that gap.

Tracker cost measurement. Kontaxis and Chew [10] measured Firefox Tracking Protection on Alexa top-200 news sites, reporting 39% data savings. The Ghostery Tracker Tax study [4] reported a ~2.2× page load speedup (19 s vs 8.6 s) on the US Alexa top-500; Castell-Uroz et al. [2] reported only ~10% bandwidth savings at 100K-site scale. Pourghassemi et al. [12] attributed CPU costs to ad scripts via Chromium activity-trace instrumentation. WhoTracks.Me [9] reports

per-site third-party content totals (median 0.42 MB) from 1.5B+ page loads. All of these works *measure* cost on completed page loads; none predicts what blocked requests would have transferred.

Predicting blocked-resource cost. The closest prior work is Brave's ad-blocker savings predictor [1]: a LASSO regression on 237 page-level features (blocked requests, DOM characteristics, JavaScript task statistics, third-party presence one-hots) for *page-level* bandwidth and JS-execution savings ($R^2 = 0.68$ for bandwidth, $R^2 = 0.73$ for CPU on a held-out 20% split, fit on 1,686 paired-crawl page loads with vs. without blocking). We differ in three respects: per-request rather than per-page granularity; no paired crawl required; explicit handling of the structurally-unobservable inference setting. A Brave-style linear regression (Ridge) on our per-request features underperforms the LUT it would replace (MAE 9,651 vs 6,802, +42%) because least-squares on a target with mean 13 KB and max 14 MB is dominated by the heavy right tail and a linear combination of pre-response features does not track the tail closely enough to cancel the squared-error penalty. Per-request blocked-resource estimation requires a loss matched to the target distribution (Fig. 4; method in §4).

3 The Tracker Cost Landscape

Before estimating the unobservable, we measure what is observable: the cost landscape ETP acts on, in three views (full distribution, by category, by lookup-key granularity). The third view sets up the granularity question §4 answers.

3.1 Data

We use the HTTP Archive June 2024 mobile crawl [5], filtering to third-party tracker requests in five categories (advertising, analytics, social, tag-manager, consent-provider) per HTTP Archive's third-party entity table. The full set is approximately 348 million tracker requests across 5,186 domains. A 1% deterministic hash sample (keyed on `page||url`) yields 3,490,824 requests across 3,723 domains, covering 92.8% of the Disconnect list's traffic-weighted footprint. Rows are split 70/15/15 train/val/test; target encodings are computed exclusively from training.

The primary target is `transfer_bytes`: on-wire response size. Its distribution is severely zero-inflated (39.5% exact zeros, mostly beacons) with a heavy right tail (max 14 MB, mean 13,607, median 43): a spike at zero and a secondary mode near 90 KB (JavaScript bundles).

The crawl that produced these labels is Chrome-based; the model is for Firefox. We show in §6 (Fig. 5 b) that a model trained on this Chrome data outperforms the deployed Firefox baseline on Firefox traffic, via two empirical tests: (i) a paired Firefox/Chrome fetch of 46 representative tracker URLs that quantifies per-URL byte agreement at the request level, and (ii) an in-page Firefox crawl that measures the model's aggregation advantage on real Firefox-served pages.

3.2 Within-domain variance

If responses from a single tracker domain were similar in size, a per-domain lookup table would suffice and there would be no need for an estimator at all. They are not (Fig. 1). We measure within-domain spread using the coefficient of variation (CV = standard deviation / mean), where CV ≈ 1 means typical responses differ from the mean by an amount comparable to the mean itself. Across 3,723 tracker domains the median CV of `transfer_bytes` is 0.94 and the 90th percentile is 3.00: for the typical domain, individual responses span an order of magnitude around the mean, and for the heaviest-tailed tenth they span three. Timing targets (TTFB, load time) by contrast have low within-domain CV (0.38) because they are determined by network conditions rather than URL identity; per-domain summarization is appropriate for them, and the rest of this paper focuses on bytes.

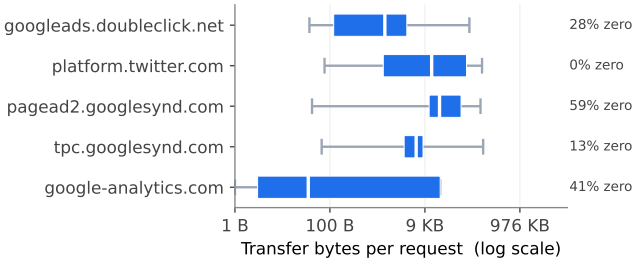

Figure 1: Within-domain transfer-size distributions for five high-volume tracker domains chosen for their wide spread. Box marks IQR (white line is median), whiskers extend to p1–p99; right column reports the share of responses that are exactly zero (beacons fired only for their server-side effect). `google-analytics.com` alone spans four orders of magnitude within its IQR.

3.3 Per-category cost share

ETP filters by Disconnect's five categories (advertising, analytics, social, tag-manager, consent-provider). We classify domains via Disconnect (canonical category by priority for multi-listed domains) plus manual additions for tag-manager infrastructure (Google Tag Manager, Tealium, Adobe DTM) Disconnect treats as non-tracker; the mapping covers 92.8% of request volume. Categories contribute unequally to blocked bandwidth (Fig. 2, Table 1): a 26-domain Tag-manager category carries 6.3% of requests but 40.6% of bytes (steepest concentration), Advertising inverts the ratio (66.5% requests, 17.8% bytes; dominated by beacons and pixels), and Content-CDN + Social contribute 25.3% and 11.0% of bytes from

heavy script and image payloads. The asymmetry motivates URL-level rather than category- or domain-level cost estimation: bytes are concentrated in a small minority of high-cost requests, and that minority is identified by URL structure within rather than across categories.

Category	*n* dom.	% req.	% bytes	Med. B
Tag manager	26	6.3	**40.6**	44
Content (CDN)	573	9.1	25.3	6,072
Advertising	2,605	**66.5**	17.8	168
Social	121	7.9	11.0	1,407
Analytics	322	8.6	3.1	43
Consent provider	38	0.7	0.8	1,220
Fingerprinting	15	0.1	0.1	7,117
Other	892	0.8	1.4	532

Table 1: Per-category share of blocked tracker traffic (HTTP Archive June 2024 mobile, full population).

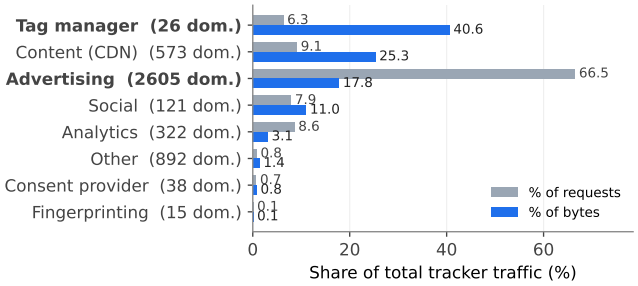

Figure 2: Share of blocked-tracker requests vs. share of bytes by Disconnect category. The 26-domain Tag-manager category is byte-heavy (41% of bytes from 6% of requests); the 2,605-domain Advertising category is byte-light (67% of requests but only 18% of bytes). Neither pattern is captured by domain- or category-level accounting; URL-level estimation is required.

3.4 From categories to a model: the path of granularity

How fine does the lookup key have to be? Fig. 3 walks the hierarchy from the deployed (domain, type) median up through intermediate keys (`has_query`, URL-length bucket, first path token) to the full URL-path LUT. Each step buys MAE at the cost of size, but the path LUT explodes to ~16M entries at full population (two orders of magnitude larger than the Disconnect list) and churns continuously (only 14.9% of unique paths in a September crawl appear in June training). Even the strongest deployable LUT we found is 25% behind the model at comparable artifact size; exact-key memorization is not a deployable primitive at any granularity that beats the model.

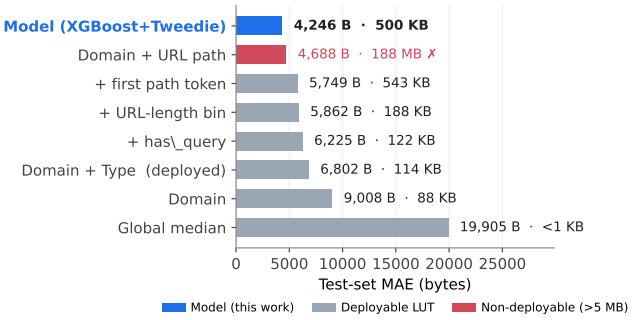

Figure 3: LUT variants vs. the model on test-set MAE; sorted ascending (best at top). Annotation gives full-population artifact size; non-deployable variants (>5 MB) marked ×. The model (blue) is within 2% of the non-deployable path LUT (350× smaller) and beats every deployable LUT we tested by at least 25%.

4 Estimating the Unobservable Response

§3.4 ruled out exact-key memorization as the right primitive. The remaining alternative is an estimator that generalizes across URL *structure*; this section builds it from features Firefox observes at block time and reports the headline against the LUT hierarchy from above. We estimate `transfer_bytes` only: Fig. 1 and the within-domain CV result (§3) show that bytes are URL-predictable in a way timing targets (TTFB, load time) are not, so URL features cannot do work for those targets that domain-level summarization does not already do.

4.1 Features and model

The features have to be reconstructible from what Firefox sees *before* a request is sent: the URL, the resource type the browser is about to request, the initiator, and the HTTP method. Within those constraints, we use 80 features in five groups, each capturing a different signal the within-domain variance result implicates: *domain priors* (2 features) carry the per-domain typical-cost baseline that the LUT also has access to; *URL structure* (12: path depth, length, query-param count, file extension) captures coarse byte-cost signals like "has a long query string" or "ends in `.js`"; *URL content* (a 50-dim TF-IDF+SVD embedding of the URL path) captures token-level patterns the structural features cannot, e.g. the model learns from training that paths matching `/gtm.js` are heavier than those matching `/pixel.gif`; *request metadata* (10: resource/initiator type, method) tells the model whether the URL was fetched as a script, image, or beacon; and *URL pattern regex flags* (6) encode a small set of hand-curated patterns we observed empirically (e.g. `register-trigger` for Privacy Sandbox).

The model is XGBoost with Tweedie loss [8, 13] (p=1.5), 500 trees, depth 8, learning rate 0.05, early stopping on validation. Two properties of the target distribution (Fig. 4) drive the loss choice. First, 39.5% of responses are exact zeros (mostly beacons fired purely for their side effect on the server); standard squared-error regression treats these as ordinary observations and pulls predictions toward them. Second, the non-zero tail is heavy: mean 13 KB, max 14 MB. Squared-error trains a model that splits the difference between the spike at zero and the heavy tail and gets neither right. Tweedie regression [13] is the standard treatment for distributions with this exact shape (a point mass at zero plus a positive continuous tail) in actuarial science, where it has been used for decades to model insurance claims; the parameter $p \in (1, 2)$ controls how aggressively errors on large-magnitude predictions are weighted relative to errors on small ones, and $p = 1.5$ is the conventional middle ground.

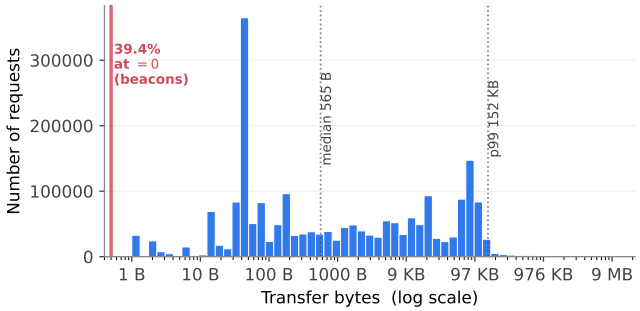

Figure 4: Distribution of `transfer_bytes` across all 3,490,824 training requests. The 39.5% spike at zero is the dominant feature (beacons fired purely for their server-side effect); the remainder is a heavy-tailed positive distribution spanning four orders of magnitude. Squared-error regression cannot fit both regimes simultaneously; Tweedie (p=1.5) is the standard treatment for distributions with this shape.

4.2 Headline: URL-level resolution achieved

The model reaches MAE 4,246 B (95% CI [4,079, 4,442], ρ=0.93) on the held-out test split (Table 2): a 37.6% MAE reduction over the deployed Domain+type LUT, and within 2% of the non-deployable path LUT despite being 350× smaller. This is the first estimator in the LUT–ladder hierarchy of §3.4 that achieves URL-level accuracy at deployable artifact size.

Path coverage decomposition. Decomposing by path coverage shows the model matches path-level memorization on the 92% of test rows the path LUT can serve (LUT 1,811 vs. model 2,264 MAE) and beats it on the remaining 8% it cannot (LUT 33,471 vs. model 27,224), at 350× smaller artifact size and with lower systematic bias that cancels in aggregation (Fig. 5 b; mechanism in §6).

Approach	Size	MAE	95% CI	ρ
Global median	<1 B	19,905	n/a	n/a
Domain LUT	88 KB	9,008	n/a	n/a
Domain+type LUT⋆	**114 KB**	**6,802**	**[6,633, 6,977]**	**0.94**
Path LUT (fallback)†	187 MB	4,326	[4,154, 4,516]	0.93
XGBoost Tweedie⋆	**500 KB**	**4,246**	**[4,079, 4,442]**	**0.93**

Table 2: Transfer-size prediction on the held-out test split. ⋆ deployable within Remote Settings envelope; † not deployable (size + continuous URL churn, §3.4). Bootstrap CIs from 1,000 resamples. The model achieves a 37.6% MAE reduction over the most accurate deployable baseline at comparable artifact size, and is within 2% of the non-deployable path-LUT upper bound.

The improvement concentrates where it matters most for aggregation: scripts and CSS, the high-bytes resource types (Table 3). Beacons and short text responses see no improvement because the LUT's near-zero prediction is already near-optimal.

Resource type	n test	D+T LUT	Model	vs. D+T
Script	144,676	12,996	**3,135**	**+75.9%**
CSS	6,142	6,222	**2,084**	+66.5%
HTML	92,438	2,246	**1,294**	+42.4%
Image	136,698	8,330	**7,428**	+10.8%
Other (beacon)	98,617	16	16	+4.6%
Text	42,984	292	303	−3.8%

Table 3: Per-resource-type MAE (bytes). Wins concentrate on script and CSS, the high-bytes classes from Table 1.

Loss and feature ablation. Off-the-shelf XGBoost defaults misfit zero-inflated web data: a squared-error baseline trails Tweedie at p=1.5 by roughly 23%, dwarfing the 4% sensitivity to the Tweedie power within $p \in [1.2, 1.8]$. URL content features compose: regex flags and TF-IDF embeddings each capture about half the URL-derived signal individually, with the combined model reaching the headline MAE in Table 2. TF-IDF subsumes most of the regex signal and adds patterns the researcher did not anticipate (e.g., Privacy Sandbox `register-trigger` paths). Laplace add-k smoothing of the path LUT does not close the gap to the model, confirming that learning URL structure beats regularizing exact-key memorization.

5 Robustness

The headline (Table 2) is computed on a random test split: rows held out from the same June 2024 crawl the model was trained on. A deployed estimator faces two further conditions the random split does not capture: (i) the world has moved on since training, and (ii) what the user actually sees is a sum over many requests, not a single prediction. We address each in turn: temporal staleness up to six months in §5.1, and aggregation behavior in §5.2. The cross-browser deployment scenario, evaluating on Firefox traffic the model was never trained on, is treated separately in §6, since it is the deployment setting itself rather than a robustness check.

5.1 Temporal stability

How stale can a deployed model be before it stops earning its keep? We train on June 2024 and evaluate on three later HTTP Archive crawls (1, 3, and 6 months out), sweeping the worst-case staleness window a quarterly-retrained estimator would experience over a full year.

Per-request MAE degrades from 4,454 (1 mo) to 5,256 (6 mo) but still beats the LUT by a quarter at 6 months; on weekly aggregation the gap shrinks more sharply (Fig. 5 a; 13 pp → 6 pp) as the model's per-request variance grows and the variance-cancellation advantage erodes. Spearman ranking remains $\rho \geq 0.92$ throughout, so even where calibration drifts the model orders requests by cost correctly. The 3-month aggregation gap is still +10.8 pp, justifying the quarterly retraining recommendation.

5.2 Aggregation: the user-facing number

Users see a weekly total, not a per-request value. *Uniform*: sample N blocked requests with replacement, sum predictions and truths, report median % error of the sum over 2,000 trials. *Domain-correlated*: first sample 15 distinct domains then N requests from those domains, modeling concentrated browsing.

	Uniform		Domain-correlated	
N	Model	D+T LUT	Model	D+T LUT
50	6.8%	20.8%	14.2%	34.5%
100	6.3%	20.9%	12.5%	34.2%
200	6.0%	21.9%	12.9%	36.4%
500	5.1%	23.2%	11.2%	36.7%

Table 4: Median weekly aggregation error. Model advantage widens under domain-correlated browsing.

6 Deployment

The estimator runs from pre-response features only, the constraint a browser-side privacy feature must satisfy. It is being integrated into a major browser vendor's privacy widget to surface concrete byte-cost numbers alongside existing tracker block-count counters.

Integration, surface, distribution. ETP's block decision is taken at Firefox's content-policy stage, where the request URI, `ContentPolicyType`, loading principal, and HTTP method are already inspected; the estimator invokes at the same hook and leaves the block decision unchanged. Per-request output sums into a per-day counter and surfaces in the New Tab widget as one additional field ("saved approximately X MB this week"). The 500 KB ONNX model plus 2 MB TF-IDF/SVD companion ship via Remote Settings, the same channel that distributes the Disconnect list and Public Suffix List; a model update is operationally identical to a Disconnect-list update.

ONNX inference performance. Single-thread per-request inference over 10,000 calls on commodity x86 (CPU-bound, synchronous, no GPU): **median 50 μs, p99 93 μs, p99.9 108 μs** via XGBoost's native predict (a conservative upper bound; the ONNX-runtime `TreeEnsembleRegressor` path is typically 2–5× faster on the same hardware). Resident memory ~5 MB, loaded once at startup; per-call allocation is a fixed-size feature vector reused across requests. The tree-count ablation places the size/accuracy elbow at 500 trees (200 trees loses the edge over the path LUT).

Deployment validation: a model trained on Chrome data wins on Firefox traffic. We close the cross-browser loop with two direct empirical tests.

Per-URL byte agreement (Claim A). A tracker's response is determined by URL and server state; URL and method are identical across browsers, cookies are per-user not per-browser, and most tracker endpoints do not UA-negotiate content. We confirm this directly: across 46 representative tracker URLs fetched under both Firefox and Chrome from the same vantage point, the median Firefox/Chrome byte ratio is 1.000 and 97.8% of URL pairs agree within 5%. The single outlier (Google reCAPTCHA at 0.63) is documented UA-conditioned bot mitigation. Per-URL bytes are browser-invariant.

In-page Firefox aggregation (Claim B). We sample crawl targets the way real browsing concentrates: the top 500 reachable hosts from the Tranco daily list dated 2026-05-03 (list ID `9W772`) [11], the high-traffic domains that dominate any real user's weekly session. Each page is crawled once with headless Firefox via Playwright using mobile emulation matching HTTP Archive's training-time conditions (360×640 viewport, mobile UA, touch, 30 s navigation timeout, fresh `BrowserContext` per page; five Firefox workers run in parallel). Of 500 attempts, 354 produced usable third-party tracker traffic after dropping navigation/bot-detection rejections and eTLD+1 first-party miscategorizations: **20,750 third-party**

tracker requests across 354 publisher pages and 1,131 distinct tracker domains, against which we run the deployed pipeline end-to-end and the deployable domain×type LUT baseline.

On weekly aggregation across this sample, the model preserves a +11 **pp to** +13 **pp** advantage over the LUT at every browsing scale (Fig. 5 b), larger than the +7 pp to +9 pp the original 35-page curated subset reported. The mechanism is the variance-cancellation effect from §5.2, amplified by the realistic browsing pattern: a LUT's per-(domain, type) bias compounds linearly across the repeated visits a real user makes to their clustered set of domains, while the model's URL-level residuals are heterogeneous enough to cancel partially in the sum. The user-facing widget surfaces that sum.

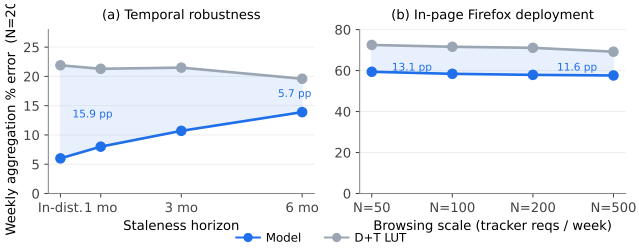

Figure 5: The model preserves its weekly-aggregation advantage over the deployable LUT across two stress axes. (a) Temporal staleness: a +13 pp advantage at 1 month erodes to +6 pp at 6 months but stays positive throughout. (b) In-page Firefox crawl on a Tranco-sampled 500-page set: +11 pp to +13 pp advantage at every browsing scale (N=50 to 500) under the realistic browsing pattern where a user revisits a clustered set of domains.

Update cadence and reproducibility. URL-path churn (§5) drives a quarterly retraining schedule via Remote Settings, bounding deployed-model staleness at three months where the aggregation advantage over the deployed baseline is still +10.8 pp (Fig. 5 a). The HTTP Archive query, sampling rule, training pipeline, ONNX artifact, Playwright crawl harness, and analysis notebooks will be released on acceptance (artifact URL withheld for double-blind compliance); the HTTP Archive dataset itself is public via BigQuery exports.

7 Implications for IETF/IRTF Privacy Measurement

Three IRTF/IETF venues have surfaced open questions this paper's measurements and methodology directly address.

MAPRG (Measurement and Analysis for Protocols RG) hosts deployed-system measurement at scale [6]. This paper's per-category bandwidth measurement and open artifact (BigQuery query, training pipeline, ONNX model, Playwright harness) contribute a tracker-blocking baseline of the type MAPRG has hosted in adjacent areas (HTTP/2 crawls, AI-crawler measurement). The pre-decision counterfactual framing (predicting what an unsent request would have transferred) is a measurement primitive that has not appeared in MAPRG sessions and may be of independent methodological interest.

HTTPbis is actively deliberating `draft-ietf-httpbis-layered-cookies-01` [14], whose own §7 admits that user-agent third-party cookie restrictions are "experiments" and that mechanism-level consensus is premature. Layered Cookies and similar drafts treat third-party content as a uniform category for restriction purposes; our per-category bandwidth shares show the category is operationally heterogeneous (41%-of-bytes Tag-manager vs. 18%-of-bytes Advertising), which is relevant input for any deployment-impact analysis.

PEARG (Privacy Enhancements and Assessments RG) maintains `draft-irtf-pearg-safe-internet-measurement` [7] as its methodology document but does not yet specify an evaluation primitive for browser-side content blockers, where the blocked artifact is by construction unobserved. Our pre-decision counterfactual recipe generalizes to any intervention that intercepts a resource before observation (Brave Shields, uBlock Origin, Safari Tracking Protection, Privacy Sandbox topic-API measurement); the Firefox deployment is a first validation point.

This paper does not propose a new protocol or RFC; it contributes a measurement methodology and a public artifact intended to fill gaps each of these groups has identified.

8 Conclusion and Future Work

Per-request cost estimation for blocked tracker requests is feasible at deployable scale. A 500 KB XGBoost+Tweedie model with 50 μs single-thread inference beats every deployable lookup-table baseline by at least 25%, comes within 2% of the non-deployable path-LUT upper bound at 350× smaller artifact size, holds its advantage out to a 6-month temporal horizon, and preserves an +11 to +13 pp aggregation advantage in domain-correlated in-page browsing across a Tranco-sampled 500-page Firefox crawl (the realistic deployment scenario for a per-user weekly counter). The methodology is being integrated into a deployed browser privacy feature. The substantive deliverable for the network-measurement community is the recipe, not the model: a Tweedie-loss treatment of zero-inflated heavy-tailed web traffic, a LUT-ladder methodology for deciding how fine the lookup key needs to

be, and an open artifact (BigQuery query, training pipeline, ONNX model, Playwright harness) that any vendor can replicate, audit, or extend to other interventions where a resource is intercepted before observation. Future work extends to cross-list robustness, cross-vendor evaluation against Brave Shields and uBlock Origin, and a Firefox-side telemetry replay against real user traffic.

References

[1] Andrius Aucinas, Istvan Haller, and Ben Livshits. 2019. Accurately Predicting Ad Blocker Savings. https://brave.com/blog/accurately-predicting-ad-blocker-savings/.

[2] Ismael Castell-Uroz, Josep Solé-Pareta, and Pere Barlet-Ros. 2020. Demystifying Content-blockers: A Large-scale Study of Actual Performance Gains. In *16th International Conference on Network and Service Management (CNSM)*. doi:10.23919/CNSM50824.2020.9269094

[3] Disconnect. 2024. Disconnect Tracking Protection List. https://github.com/disconnectme/disconnect-tracking-protection. Accessed: 2024-06-01.

[4] Ghostery. 2018. The Tracker Tax: The Impact of Third-Party Trackers on Website Speed in the United States. https://0x65.dev/static/docs/studies/tracker_tax.pdf.

[5] HTTP Archive. 2024. HTTP Archive: Tracking How the Web is Built. https://httparchive.org.

[6] Internet Research Task Force, Measurement and Analysis for Protocols Research Group. 2025. MAPRG agenda, IETF 124. https://datatracker.ietf.org/doc/agenda-124-maprg/.

[7] Internet Research Task Force, Privacy Enhancements and Assessments Research Group. 2026. Guidelines for Performing Safe Measurement on the Internet. Internet-Draft draft-irtf-pearg-safe-internet-measurement-14. Work in progress, expires 29 July 2026.

[8] Bent Jørgensen. 1987. Exponential Dispersion Models. *Journal of the Royal Statistical Society: Series B (Methodological)* 49, 2 (1987), 127–145.

[9] Arjaldo Karaj, Sam Macbeth, Rémi Berson, and Josep M Pujol. 2018. WhoTracks.Me: Shedding light on the opaque world of online tracking. arXiv preprint arXiv:1804.08959.

[10] Georgios Kontaxis and Monica Chew. 2015. Tracking Protection in Firefox for Privacy and Performance. In *IEEE Workshop on Web 2.0 Security and Privacy (W2SP)*.

[11] Victor Le Pochat, Tom Van Goethem, Samaneh Tajalizadehkhoob, Maciej Korczyński, and Wouter Joosen. 2019. Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation. In *Proceedings of the 26th Annual Network and Distributed System Security Symposium (NDSS)*. doi:10.14722/ndss.2019.23386

[12] Behnam Pourghassemi, Jordan Bonecutter, Zhou Li, and Aparna Chandramowlishwaran. 2021. adPerf: Characterizing the Performance of Third-party Ads. *Proceedings of the ACM on Measurement and Analysis of Computing Systems (POMACS)* 5, 1, Article 8 (2021). doi:10.1145/3447381

[13] Gordon K Smyth and Bent Jørgensen. 2002. Fitting Tweedie's Compound Poisson Model to Insurance Claims Data: Dispersion Modelling. *ASTIN Bulletin* 32, 1 (2002), 143–157.

[14] Anne van Kesteren and Johann Hofmann. 2025. Layered Cookies. Internet-Draft draft-ietf-httpbis-layered-cookies-01. Work in progress.